Subdivision Algoritmus Tesztelés Processingben

https://gemini.google.com/app/ec5d3d2f804e098b

User prompt: generálj új kódot, amiben a catmul clark algoritmust írod meg egy téglatestre 1-es gomb megnyomására növeld eggyel a lefutások számát, 2-es gomb menyomására csökkentsd eggyel a lefutások számát

Response: Íme a teljes Processing kód, amely egy téglatesten valósítja meg a **Catmull-Clark subdivision** algoritmust.

A kódban az **1-es gomb** megnyomásával tudod növelni (egészen 4-es szintig, hogy ne lassuljon le nagyon), a **2-es gombbal** pedig csökkenteni az osztások számát. A kiinduló téglatest automatikusan újraszámolódik a megadott szintek alapján.

```
import java.util.HashMap;
import java.util.ArrayList;

ArrayList<PVector> baseVertices;
ArrayList<ArrayList<Integer>> baseFaces;

ArrayList<PVector> currentVertices;
ArrayList<ArrayList<Integer>> currentFaces;
int subLevel = 0;

void settings() {
  size(800, 600, P3D);
}

void setup() {
  surface.setLocation(100, 100);
  initBox(160, 100, 120);
  rebuildMesh();
}

void initBox(float w, float h, float d) {
  baseVertices = new ArrayList<PVector>();
  // 8 sarkú téglatest csúcspontjai
  baseVertices.add(new PVector(-w/2, -h/2, -d/2)); // 0
  baseVertices.add(new PVector( w/2, -h/2, -d/2)); // 1
  baseVertices.add(new PVector( w/2,  h/2, -d/2)); // 2
  baseVertices.add(new PVector(-w/2,  h/2, -d/2)); // 3
  baseVertices.add(new PVector(-w/2, -h/2,  d/2)); // 4
  baseVertices.add(new PVector( w/2, -h/2,  d/2)); // 5
```

```
  baseVertices.add(new PVector( w/2,  h/2,  d/2)); // 6
  baseVertices.add(new PVector(-w/2,  h/2,  d/2)); // 7

  baseFaces = new ArrayList<ArrayList<Integer>>();
  addBaseFace(0, 3, 2, 1); // Hátlap
  addBaseFace(4, 5, 6, 7); // Előlap
  addBaseFace(0, 1, 5, 4); // Tetőlap
  addBaseFace(2, 3, 7, 6); // Aljzat
  addBaseFace(0, 4, 7, 3); // Bal oldal
  addBaseFace(1, 2, 6, 5); // Jobb oldal
}

void addBaseFace(int a, int b, int c, int d) {
  ArrayList<Integer> f = new ArrayList<Integer>();
  f.add(a); f.add(b); f.add(c); f.add(d);
  baseFaces.add(f);
}

void keyPressed() {
  if (key == '1') {
    subLevel = constrain(subLevel + 1, 0, 4);
    rebuildMesh();
  } else if (key == '2') {
    subLevel = constrain(subLevel - 1, 0, 4);
    rebuildMesh();
  }
}

void rebuildMesh() {
  currentVertices = new ArrayList<PVector>();
  for (PVector v : baseVertices) currentVertices.add(v.copy());

  currentFaces = new ArrayList<ArrayList<Integer>>();
  for (ArrayList<Integer> f : baseFaces) {
    ArrayList<Integer> nf = new ArrayList<Integer>();
    for (int idx : f) nf.add(idx);
    currentFaces.add(nf);
  }

  for (int i = 0; i < subLevel; i++) {
    catmullClarkStep();
  }
}

void catmullClarkStep() {
  int numVerts = currentVertices.size();
  int numFaces = currentFaces.size();

  // 1. Lapok középpontjai (Face Points)
  ArrayList<PVector> facePoints = new ArrayList<PVector>();
  for (int i = 0; i < numFaces; i++) {
    ArrayList<Integer> f = currentFaces.get(i);
```

```
    PVector fp = new PVector(0, 0, 0);
    for (int idx : f) {
      fp.add(currentVertices.get(idx));
    }
    fp.div(f.size());
    facePoints.add(fp);
  }

  // 2. Élek és élek pontjai (Edge Points)
  HashMap<String, ArrayList<Integer>> edgeFaces = new HashMap<String,
ArrayList<Integer>>();
  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int size = f.size();
    for (int i = 0; i < size; i++) {
      int u = f.get(i);
      int v = f.get((i + 1) % size);
      String key = Math.min(u, v) + "_" + Math.max(u, v);
      if (!edgeFaces.containsKey(key)) {
        edgeFaces.put(key, new ArrayList<Integer>());
      }
      edgeFaces.get(key).add(fIdx);
    }
  }

  HashMap<String, PVector> edgePoints = new HashMap<String, PVector>();
  for (String key : edgeFaces.keySet()) {
    String[] parts = key.split("_");
    int u = Integer.parseInt(parts[0]);
    int v = Integer.parseInt(parts[1]);
    PVector p1 = currentVertices.get(u);
    PVector p2 = currentVertices.get(v);

    PVector ep = new PVector(0, 0, 0);
    ep.add(p1);
    ep.add(p2);
    ArrayList<Integer> adjFaces = edgeFaces.get(key);
    for (int fIdx : adjFaces) {
      ep.add(facePoints.get(fIdx));
    }
    ep.div(2.0f + adjFaces.size());
    edgePoints.put(key, ep);
  }

  // 3. Eredeti csúcsok új pozíciói (Vertex Points)
  ArrayList<ArrayList<Integer>> vertexFaces = new
ArrayList<ArrayList<Integer>>();
  ArrayList<ArrayList<String>> vertexEdges = new
ArrayList<ArrayList<String>>();
  for (int i = 0; i < numVerts; i++) {
    vertexFaces.add(new ArrayList<Integer>());
    vertexEdges.add(new ArrayList<String>());
```

```
  }

  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    for (int idx : f) {
      if (!vertexFaces.get(idx).contains(fIdx)) {
        vertexFaces.get(idx).add(fIdx);
      }
    }
  }

  for (String key : edgeFaces.keySet()) {
    String[] parts = key.split("_");
    int u = Integer.parseInt(parts[0]);
    int v = Integer.parseInt(parts[1]);
    if (!vertexEdges.get(u).contains(key)) vertexEdges.get(u).add(key);
    if (!vertexEdges.get(v).contains(key)) vertexEdges.get(v).add(key);
  }

  ArrayList<PVector> newVertexPositions = new ArrayList<PVector>();
  for (int i = 0; i < numVerts; i++) {
    PVector oldV = currentVertices.get(i);
    ArrayList<Integer> vFaces = vertexFaces.get(i);
    ArrayList<String> vEdges = vertexEdges.get(i);
    int n = vEdges.size();

    if (n == 0) {
      newVertexPositions.add(oldV.copy());
      continue;
    }

    PVector Q = new PVector(0, 0, 0);
    for (int fIdx : vFaces) {
      Q.add(facePoints.get(fIdx));
    }
    Q.div(vFaces.size());

    PVector R = new PVector(0, 0, 0);
    for (String eKey : vEdges) {
      String[] parts = eKey.split("_");
      int u = Integer.parseInt(parts[0]);
      int v = Integer.parseInt(parts[1]);
      PVector other = (u == i) ? currentVertices.get(v) :
currentVertices.get(u);
      PVector mid = PVector.add(oldV, other).mult(0.5f);
      R.add(mid);
    }
    R.div(n);

    PVector newV = new PVector(0, 0, 0);
    newV.add(Q);
    newV.add(PVector.mult(R, 2.0f));
```

```
    newV.add(PVector.mult(oldV, n - 3));
    newV.div(n);

    newVertexPositions.add(newV);
  }

  // 4. Új topológia összeállítása
  ArrayList<PVector> nextVertices = new ArrayList<PVector>();
  for (PVector v : newVertexPositions) {
    nextVertices.add(v);
  }

  int facePointStartIndex = nextVertices.size();
  for (PVector fp : facePoints) {
    nextVertices.add(fp);
  }

  HashMap<String, Integer> edgePointIndices = new HashMap<String, Integer>();
  for (String key : edgePoints.keySet()) {
    edgePointIndices.put(key, nextVertices.size());
    nextVertices.add(edgePoints.get(key));
  }

  ArrayList<ArrayList<Integer>> nextFaces = new
ArrayList<ArrayList<Integer>>();
  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int fpIdx = facePointStartIndex + fIdx;
    int size = f.size();
    for (int i = 0; i < size; i++) {
      int currV = f.get(i);
      int nextV = f.get((i + 1) % size);
      int prevV = f.get((i - 1 + size) % size);

      String edgeCurr = Math.min(currV, nextV) + "_" + Math.max(currV, nextV);
      String edgePrev = Math.min(currV, prevV) + "_" + Math.max(currV, prevV);

      int epCurr = edgePointIndices.get(edgeCurr);
      int epPrev = edgePointIndices.get(edgePrev);

      ArrayList<Integer> newQuad = new ArrayList<Integer>();
      newQuad.add(currV);
      newQuad.add(epCurr);
      newQuad.add(fpIdx);
      newQuad.add(epPrev);
      nextFaces.add(newQuad);
    }
  }

  currentVertices = nextVertices;
  currentFaces = nextFaces;
}
```

```
void draw() {
  background(30);
  lights();

  translate(width / 2, height / 2, 0);
  rotateX(frameCount * 0.005);
  rotateY(frameCount * 0.008);

  stroke(100, 200, 255);
  fill(40, 90, 140, 220);

  for (ArrayList<Integer> f : currentFaces) {
    beginShape();
    for (int idx : f) {
      PVector v = currentVertices.get(idx);
      vertex(v.x, v.y, v.z);
    }
    endShape(CLOSE);
  }

  // Információs szöveg rajzolása a képernyőre
  hint(DISABLE_DEPTH_TEST);
  fill(255);
  textSize(16);
  text("Subdivision szint: " + subLevel, -width/2 + 20, -height/2 + 30);
  text("1-es gomb: növelés | 2-es gomb: csökkentés", -width/2 + 20, -height/2 +
55);
  hint(ENABLE_DEPTH_TEST);
}
```

Szeretnéd, hogy az éleket is kirajzoljuk rácsként (wireframe módban) a jobban láthatóság érdekében?

User prompt: Módosítsd a kódot, hogy az egérrel tudjam mozgatni és a szöveg ne mozogjon el a térben

Response: Íme a módosított kód! Beépítettem az egérrel történő forgatást (mouseDragged segítségével), a 3D objektumot pedig egy pushMatrix() és popMatrix() blokk közé zártam. Emiatt a szöveg (HUD) teljesen fixen marad a képernyő bal felső sarkában, és nem fog együtt forogni a téglatesttel.

```
import java.util.HashMap;
import java.util.ArrayList;

ArrayList<PVector> baseVertices;
ArrayList<ArrayList<Integer>> baseFaces;

ArrayList<PVector> currentVertices;
ArrayList<ArrayList<Integer>> currentFaces;
int subLevel = 0;
```

```
// Forgatási szögek az egérkezeléshez
float rotX = 0;
float rotY = 0;

void settings() {
  size(800, 600, P3D);
}

void setup() {
  surface.setLocation(100, 100);
  initBox(160, 100, 120);
  rebuildMesh();
}

void initBox(float w, float h, float d) {
  baseVertices = new ArrayList<PVector>();
  baseVertices.add(new PVector(-w/2, -h/2, -d/2)); // 0
  baseVertices.add(new PVector( w/2, -h/2, -d/2)); // 1
  baseVertices.add(new PVector( w/2,  h/2, -d/2)); // 2
  baseVertices.add(new PVector(-w/2,  h/2, -d/2)); // 3
  baseVertices.add(new PVector(-w/2, -h/2,  d/2)); // 4
  baseVertices.add(new PVector( w/2, -h/2,  d/2)); // 5
  baseVertices.add(new PVector( w/2,  h/2,  d/2)); // 6
  baseVertices.add(new PVector(-w/2,  h/2,  d/2)); // 7

  baseFaces = new ArrayList<ArrayList<Integer>>();
  addBaseFace(0, 3, 2, 1); // Hátlap
  addBaseFace(4, 5, 6, 7); // Előlap
  addBaseFace(0, 1, 5, 4); // Tetőlap
  addBaseFace(2, 3, 7, 6); // Aljzat
  addBaseFace(0, 4, 7, 3); // Bal oldal
  addBaseFace(1, 2, 6, 5); // Jobb oldal
}

void addBaseFace(int a, int b, int c, int d) {
  ArrayList<Integer> f = new ArrayList<Integer>();
  f.add(a); f.add(b); f.add(c); f.add(d);
  baseFaces.add(f);
}

void keyPressed() {
  if (key == '1') {
    subLevel = constrain(subLevel + 1, 0, 4);
    rebuildMesh();
  } else if (key == '2') {
    subLevel = constrain(subLevel - 1, 0, 4);
    rebuildMesh();
  }
}

// Egérmozgatás figyelése a forgatáshoz
```

```
void mouseDragged() {
  float dx = mouseX - pmouseX;
  float dy = mouseY - pmouseY;
  rotY += dx * 0.01f;
  rotX -= dy * 0.01f;
}

void rebuildMesh() {
  currentVertices = new ArrayList<PVector>();
  for (PVector v : baseVertices) currentVertices.add(v.copy());

  currentFaces = new ArrayList<ArrayList<Integer>>();
  for (ArrayList<Integer> f : baseFaces) {
    ArrayList<Integer> nf = new ArrayList<Integer>();
    for (int idx : f) nf.add(idx);
    currentFaces.add(nf);
  }

  for (int i = 0; i < subLevel; i++) {
    catmullClarkStep();
  }
}

void catmullClarkStep() {
  int numVerts = currentVertices.size();
  int numFaces = currentFaces.size();

  // 1. Lapok középpontjai (Face Points)
  ArrayList<PVector> facePoints = new ArrayList<PVector>();
  for (int i = 0; i < numFaces; i++) {
    ArrayList<Integer> f = currentFaces.get(i);
    PVector fp = new PVector(0, 0, 0);
    for (int idx : f) {
      fp.add(currentVertices.get(idx));
    }
    fp.div(f.size());
    facePoints.add(fp);
  }

  // 2. Élek és élek pontjai (Edge Points)
  HashMap<String, ArrayList<Integer>> edgeFaces = new HashMap<String,
ArrayList<Integer>>();
  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int size = f.size();
    for (int i = 0; i < size; i++) {
      int u = f.get(i);
      int v = f.get((i + 1) % size);
      String key = Math.min(u, v) + "_" + Math.max(u, v);
      if (!edgeFaces.containsKey(key)) {
        edgeFaces.put(key, new ArrayList<Integer>());
      }
```

```
      edgeFaces.get(key).add(fIdx);
    }
  }

  HashMap<String, PVector> edgePoints = new HashMap<String, PVector>();
  for (String key : edgeFaces.keySet()) {
    String[] parts = key.split("_");
    int u = Integer.parseInt(parts[0]);
    int v = Integer.parseInt(parts[1]);
    PVector p1 = currentVertices.get(u);
    PVector p2 = currentVertices.get(v);

    PVector ep = new PVector(0, 0, 0);
    ep.add(p1);
    ep.add(p2);
    ArrayList<Integer> adjFaces = edgeFaces.get(key);
    for (int fIdx : adjFaces) {
      ep.add(facePoints.get(fIdx));
    }
    ep.div(2.0f + adjFaces.size());
    edgePoints.put(key, ep);
  }

  // 3. Eredeti csúcsok új pozíciói (Vertex Points)
  ArrayList<ArrayList<Integer>> vertexFaces = new
ArrayList<ArrayList<Integer>>();
  ArrayList<ArrayList<String>> vertexEdges = new
ArrayList<ArrayList<String>>();
  for (int i = 0; i < numVerts; i++) {
    vertexFaces.add(new ArrayList<Integer>());
    vertexEdges.add(new ArrayList<String>());
  }

  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    for (int idx : f) {
      if (!vertexFaces.get(idx).contains(fIdx)) {
        vertexFaces.get(idx).add(fIdx);
      }
    }
  }

  for (String key : edgeFaces.keySet()) {
    String[] parts = key.split("_");
    int u = Integer.parseInt(parts[0]);
    int v = Integer.parseInt(parts[1]);
    if (!vertexEdges.get(u).contains(key)) vertexEdges.get(u).add(key);
    if (!vertexEdges.get(v).contains(key)) vertexEdges.get(v).add(key);
  }

  ArrayList<PVector> newVertexPositions = new ArrayList<PVector>();
  for (int i = 0; i < numVerts; i++) {
```

```
    PVector oldV = currentVertices.get(i);
    ArrayList<Integer> vFaces = vertexFaces.get(i);
    ArrayList<String> vEdges = vertexEdges.get(i);
    int n = vEdges.size();

    if (n == 0) {
      newVertexPositions.add(oldV.copy());
      continue;
    }

    PVector Q = new PVector(0, 0, 0);
    for (int fIdx : vFaces) {
      Q.add(facePoints.get(fIdx));
    }
    Q.div(vFaces.size());

    PVector R = new PVector(0, 0, 0);
    for (String eKey : vEdges) {
      String[] parts = eKey.split("_");
      int u = Integer.parseInt(parts[0]);
      int v = Integer.parseInt(parts[1]);
      PVector other = (u == i) ? currentVertices.get(v) :
currentVertices.get(u);
      PVector mid = PVector.add(oldV, other).mult(0.5f);
      R.add(mid);
    }
    R.div(n);

    PVector newV = new PVector(0, 0, 0);
    newV.add(Q);
    newV.add(PVector.mult(R, 2.0f));
    newV.add(PVector.mult(oldV, n - 3));
    newV.div(n);

    newVertexPositions.add(newV);
  }

  // 4. Új topológia összeállítása
  ArrayList<PVector> nextVertices = new ArrayList<PVector>();
  for (PVector v : newVertexPositions) {
    nextVertices.add(v);
  }

  int facePointStartIndex = nextVertices.size();
  for (PVector fp : facePoints) {
    nextVertices.add(fp);
  }

  HashMap<String, Integer> edgePointIndices = new HashMap<String, Integer>();
  for (String key : edgePoints.keySet()) {
    edgePointIndices.put(key, nextVertices.size());
    nextVertices.add(edgePoints.get(key));
```

```
  }

  ArrayList<ArrayList<Integer>> nextFaces = new
ArrayList<ArrayList<Integer>>();
  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int fpIdx = facePointStartIndex + fIdx;
    int size = f.size();
    for (int i = 0; i < size; i++) {
      int currV = f.get(i);
      int nextV = f.get((i + 1) % size);
      int prevV = f.get((i - 1 + size) % size);

      String edgeCurr = Math.min(currV, nextV) + "_" + Math.max(currV, nextV);
      String edgePrev = Math.min(currV, prevV) + "_" + Math.max(currV, prevV);

      int epCurr = edgePointIndices.get(edgeCurr);
      int epPrev = edgePointIndices.get(edgePrev);

      ArrayList<Integer> newQuad = new ArrayList<Integer>();
      newQuad.add(currV);
      newQuad.add(epCurr);
      newQuad.add(fpIdx);
      newQuad.add(epPrev);
      nextFaces.add(newQuad);
    }
  }

  currentVertices = nextVertices;
  currentFaces = nextFaces;
}

void draw() {
  background(30);
  lights();

  // --- 3D TÉR (Csak a test forgatásához) ---
  pushMatrix();
  translate(width / 2, height / 2, 0);
  rotateX(rotX);
  rotateY(rotY);

  stroke(100, 200, 255);
  fill(40, 90, 140, 220);

  for (ArrayList<Integer> f : currentFaces) {
    beginShape();
    for (int idx : f) {
      PVector v = currentVertices.get(idx);
      vertex(v.x, v.y, v.z);
    }
    endShape(CLOSE);
```

```
  }
  popMatrix();

  // --- 2D HUD / SZÖVEG (Fixen marad a sarokban) ---
  hint(DISABLE_DEPTH_TEST);
  fill(255);
  textSize(16);
  text("Subdivision szint: " + subLevel, 20, 30);
  text("1-es gomb: növelés | 2-es gomb: csökkentés", 20, 55);
  text("Forgatás: kattints és húzd az egeret", 20, 80);
  hint(ENABLE_DEPTH_TEST);
}
```

Szeretnéd, hogy a modell drótvázas (wireframe) megjelenítését is bekapcsoljuk, vagy így tökéletes?

User prompt: Módosítsd, hogy be tudjak tölteni obj fájlokat és arra is működjön az algoritmus

Response: Íme a frissített kód! Módosítottam úgy, hogy be tudjon olvasni bármilyen **Wavefront .obj fájlt** a Processing data mappájából (például a korábban elmentett cube.obj vagy sphere.obj fájlt), majd arra futtassa le a Catmull-Clark subdivision algoritmust.

A kódban beállítottam az autómatikus skálázást is, így a modell pontosan az ablak közepén fog megjelenni megfelelő méretben, függetlenül attól, hogy mekkora koordinátákkal bír az OBJ fájl.

```
import java.util.HashMap;
import java.util.ArrayList;

ArrayList<PVector> baseVertices;
ArrayList<ArrayList<Integer>> baseFaces;

ArrayList<PVector> currentVertices;
ArrayList<ArrayList<Integer>> currentFaces;
int subLevel = 0;

// Forgatási szögek az egérkezeléshez
float rotX = 0;
float rotY = 0;

void settings() {
  size(800, 600, P3D);
}

void setup() {
  surface.setLocation(100, 100);

  // Tölts be egy OBJ fájlt a data mappából (pl. "cube.obj" vagy "sphere.obj")
  loadOBJ("cube.obj");
```

```
  rebuildMesh();
}

void loadOBJ(String filename) {
  baseVertices = new ArrayList<PVector>();
  baseFaces = new ArrayList<ArrayList<Integer>>();

  String[] lines = loadStrings(filename);
  if (lines == null) {
    println("Hiba: Nem található a fájl: " + filename);
    return;
  }

  for (String line : lines) {
    line = line.trim();
    if (line.startsWith("v ")) {
      // Csúcspont (vertex) beolvasása
      String[] parts = line.split("\\s+");
      float x = float(parts[1]);
      float y = float(parts[2]);
      float z = float(parts[3]);
      baseVertices.add(new PVector(x, y, z));
    } else if (line.startsWith("f ")) {
      // Lap (face) beolvasása
      String[] parts = line.split("\\s+");
      ArrayList<Integer> face = new ArrayList<Integer>();
      for (int i = 1; i < parts.length; i++) {
        String[] indices = parts[i].split("/");
        // Az OBJ fájlok 1-alapú indexelést használnak, ezt konvertáljuk
0-alapúra
        int vIndex = Integer.parseInt(indices[0]) - 1;
        face.add(vIndex);
      }
      baseFaces.add(face);
    }
  }

  // A modell automatikus középre igazítása és méretezése
  centerAndScaleModel();
}

void centerAndScaleModel() {
  if (baseVertices.isEmpty()) return;

  // Súlypont kiszámítása
  PVector center = new PVector(0, 0, 0);
  for (PVector v : baseVertices) {
    center.add(v);
  }
  center.div(baseVertices.size());

  // Középre tolás
```

```
  for (PVector v : baseVertices) {
    v.sub(center);
  }

  // Legnagyobb távolság keresése a skálázáshoz
  float maxDist = 0;
  for (PVector v : baseVertices) {
    float d = v.mag();
    if (d > maxDist) maxDist = d;
  }

  // Skálázás egy kényelmes méretre (~150 pixel sugarú)
  if (maxDist > 0) {
    float scaleFactor = 150.0f / maxDist;
    for (PVector v : baseVertices) {
      v.mult(scaleFactor);
    }
  }
}

void keyPressed() {
  if (key == '1') {
    subLevel = constrain(subLevel + 1, 0, 4);
    rebuildMesh();
  } else if (key == '2') {
    subLevel = constrain(subLevel - 1, 0, 4);
    rebuildMesh();
  }
}

void mouseDragged() {
  float dx = mouseX - pmouseX;
  float dy = mouseY - pmouseY;
  rotY += dx * 0.01f;
  rotX -= dy * 0.01f;
}

void rebuildMesh() {
  currentVertices = new ArrayList<PVector>();
  for (PVector v : baseVertices) currentVertices.add(v.copy());

  currentFaces = new ArrayList<ArrayList<Integer>>();
  for (ArrayList<Integer> f : baseFaces) {
    ArrayList<Integer> nf = new ArrayList<Integer>();
    for (int idx : f) nf.add(idx);
    currentFaces.add(nf);
  }

  for (int i = 0; i < subLevel; i++) {
    catmullClarkStep();
  }
}
```

```
void catmullClarkStep() {
  int numVerts = currentVertices.size();
  int numFaces = currentFaces.size();

  // 1. Lapok középpontjai (Face Points)
  ArrayList<PVector> facePoints = new ArrayList<PVector>();
  for (int i = 0; i < numFaces; i++) {
    ArrayList<Integer> f = currentFaces.get(i);
    PVector fp = new PVector(0, 0, 0);
    for (int idx : f) {
      fp.add(currentVertices.get(idx));
    }
    fp.div(f.size());
    facePoints.add(fp);
  }

  // 2. Élek és élek pontjai (Edge Points)
  HashMap<String, ArrayList<Integer>> edgeFaces = new HashMap<String,
ArrayList<Integer>>();
  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int size = f.size();
    for (int i = 0; i < size; i++) {
      int u = f.get(i);
      int v = f.get((i + 1) % size);
      String key = Math.min(u, v) + "_" + Math.max(u, v);
      if (!edgeFaces.containsKey(key)) {
        edgeFaces.put(key, new ArrayList<Integer>());
      }
      edgeFaces.get(key).add(fIdx);
    }
  }

  HashMap<String, PVector> edgePoints = new HashMap<String, PVector>();
  for (String key : edgeFaces.keySet()) {
    String[] parts = key.split("_");
    int u = Integer.parseInt(parts[0]);
    int v = Integer.parseInt(parts[1]);
    PVector p1 = currentVertices.get(u);
    PVector p2 = currentVertices.get(v);

    PVector ep = new PVector(0, 0, 0);
    ep.add(p1);
    ep.add(p2);
    ArrayList<Integer> adjFaces = edgeFaces.get(key);
    for (int fIdx : adjFaces) {
      ep.add(facePoints.get(fIdx));
    }
    ep.div(2.0f + adjFaces.size());
    edgePoints.put(key, ep);
  }
```

```
  // 3. Eredeti csúcsok új pozíciói (Vertex Points)
  ArrayList<ArrayList<Integer>> vertexFaces = new
ArrayList<ArrayList<Integer>>();
  ArrayList<ArrayList<String>> vertexEdges = new
ArrayList<ArrayList<String>>();
  for (int i = 0; i < numVerts; i++) {
    vertexFaces.add(new ArrayList<Integer>());
    vertexEdges.add(new ArrayList<String>());
  }

  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    for (int idx : f) {
      if (!vertexFaces.get(idx).contains(fIdx)) {
        vertexFaces.get(idx).add(fIdx);
      }
    }
  }

  for (String key : edgeFaces.keySet()) {
    String[] parts = key.split("_");
    int u = Integer.parseInt(parts[0]);
    int v = Integer.parseInt(parts[1]);
    if (!vertexEdges.get(u).contains(key)) vertexEdges.get(u).add(key);
    if (!vertexEdges.get(v).contains(key)) vertexEdges.get(v).add(key);
  }

  ArrayList<PVector> newVertexPositions = new ArrayList<PVector>();
  for (int i = 0; i < numVerts; i++) {
    PVector oldV = currentVertices.get(i);
    ArrayList<Integer> vFaces = vertexFaces.get(i);
    ArrayList<String> vEdges = vertexEdges.get(i);
    int n = vEdges.size();

    if (n == 0) {
      newVertexPositions.add(oldV.copy());
      continue;
    }

    PVector Q = new PVector(0, 0, 0);
    for (int fIdx : vFaces) {
      Q.add(facePoints.get(fIdx));
    }
    Q.div(vFaces.size());

    PVector R = new PVector(0, 0, 0);
    for (String eKey : vEdges) {
      String[] parts = eKey.split("_");
      int u = Integer.parseInt(parts[0]);
      int v = Integer.parseInt(parts[1]);
      PVector other = (u == i) ? currentVertices.get(v) :
```

```
currentVertices.get(u);
      PVector mid = PVector.add(oldV, other).mult(0.5f);
      R.add(mid);
    }
    R.div(n);

    PVector newV = new PVector(0, 0, 0);
    newV.add(Q);
    newV.add(PVector.mult(R, 2.0f));
    newV.add(PVector.mult(oldV, n - 3));
    newV.div(n);

    newVertexPositions.add(newV);
  }

  // 4. Új topológia összeállítása
  ArrayList<PVector> nextVertices = new ArrayList<PVector>();
  for (PVector v : newVertexPositions) {
    nextVertices.add(v);
  }

  int facePointStartIndex = nextVertices.size();
  for (PVector fp : facePoints) {
    nextVertices.add(fp);
  }

  HashMap<String, Integer> edgePointIndices = new HashMap<String, Integer>();
  for (String key : edgePoints.keySet()) {
    edgePointIndices.put(key, nextVertices.size());
    nextVertices.add(edgePoints.get(key));
  }

  ArrayList<ArrayList<Integer>> nextFaces = new
ArrayList<ArrayList<Integer>>();
  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int fpIdx = facePointStartIndex + fIdx;
    int size = f.size();
    for (int i = 0; i < size; i++) {
      int currV = f.get(i);
      int nextV = f.get((i + 1) % size);
      int prevV = f.get((i - 1 + size) % size);

      String edgeCurr = Math.min(currV, nextV) + "_" + Math.max(currV, nextV);
      String edgePrev = Math.min(currV, prevV) + "_" + Math.max(currV, prevV);

      int epCurr = edgePointIndices.get(edgeCurr);
      int epPrev = edgePointIndices.get(edgePrev);

      ArrayList<Integer> newQuad = new ArrayList<Integer>();
      newQuad.add(currV);
      newQuad.add(epCurr);
```

```
      newQuad.add(fpIdx);
      newQuad.add(epPrev);
      nextFaces.add(newQuad);
    }
  }

  currentVertices = nextVertices;
  currentFaces = nextFaces;
}

void draw() {
  background(30);
  lights();

  // --- 3D TÉR (Forgatás) ---
  pushMatrix();
  translate(width / 2, height / 2, 0);
  rotateX(rotX);
  rotateY(rotY);

  stroke(100, 200, 255);
  fill(40, 90, 140, 220);

  for (ArrayList<Integer> f : currentFaces) {
    beginShape();
    for (int idx : f) {
      PVector v = currentVertices.get(idx);
      vertex(v.x, v.y, v.z);
    }
    endShape(CLOSE);
  }
  popMatrix();

  // --- 2D HUD (Szöveg) ---
  hint(DISABLE_DEPTH_TEST);
  fill(255);
  textSize(16);
  text("Subdivision szint: " + subLevel, 20, 30);
  text("1-es gomb: növelés | 2-es gomb: csökkentés", 20, 55);
  text("Forgatás: kattints és húzd az egeret", 20, 80);
  hint(ENABLE_DEPTH_TEST);
}
```

Használat:

1. Biztosítsd, hogy a kért OBJ fájl (pl. cube.obj vagy sphere.obj) a skicced **data** mappájában van.
2. A setup() függvényben módosítsd a loadOBJ("cube.obj"); sort arra a fájlnévre, amit meg szeretnél nyitni (pl. loadOBJ("sphere.obj");).
